Scrum Poker – Fragenkatalog fürs Fachgespräch

Teil 03 der Vorbereitung (Übersicht und Lernplan: README). **120 wahrscheinliche Fragen** mit kurzen Musterantworten. Teil A ist nach den Foliensätzen geordnet, Teil B nach den typischen Frageformen zum Projekt.

So übt ihr: Zu zweit. Eine Person liest die Frage vor, die andere antwortet **mit dem Code auf dem Bildschirm** und zeigt die Stelle (`Ctrl+G` springt zur Zeile). Erst danach die Musterantwort lesen. Antworten nie auswendig lernen, sondern in eigenen Worten sagen. Wer eine Frage nicht beantworten kann, liest die Stelle in Teil 01 (Client) bzw. Teil 02 (Server) Zeile für Zeile nach und markiert die Frage für die nächste Runde.

Verweise: `client/src/components/App.tsx:11` = Datei und Zeile, Pfade ab dem Projektordner. «09/30» = Foliensatz 09, Folie 30. «Nicht in den Folien» heisst: kommt im Kurs (noch) nicht vor, wir können es aber begründen.

Inhalt

- Teil A – Fragen nach Folien-Themen
  - A.1 Web-Grundlagen und Architektur (01, 02)
  - A.2 Node.js, npm, pnpm (03)
  - A.3 Server: Express und REST (01/4, 02/4, 02/10-11)
  - A.4 HTML und CSS (04)
  - A.5 Responsive Design (05)
  - A.6 TypeScript Basics (06)
  - A.7 TypeScript Advanced (07)
  - A.8 React Basics (08)
  - A.9 React Hooks (09)
- Teil B – Fragen zu unserem Projekt
  - B.1 «Zeig mir, wo …»
  - B.2 «Was passiert, wenn …»
  - B.3 «Warum habt ihr … statt …?»
  - B.4 «Was würdet ihr ändern?»
  - B.5 KI-Nutzung

Teil A – Fragen nach Folien-Themen

A.1 Web-Grundlagen und Architektur (01, 02)

1. Beschreibt die Architektur eurer App in drei Sätzen. Ein React-Client im Browser (Presentation) spricht über HTTP mit einem Express-Server (Web-Server und Application in einem, Folie 02/4). Der Server hält alle Sessions in einem Array im Arbeitsspeicher und bietet eine REST-API unter `/api/sessions` an. Der Client fragt den Zustand regelmässig ab (Polling) und schickt jede Aktion als eigenen Request.

2. Was ist eine SPA, und ist eure App eine? Single Page Application: Es gibt eine einzige HTML-Seite (`client/index.html:10-11` mit `<div id="root">`), React tauscht danach nur Teile der

Seite aus. Ja: Der Wechsel zwischen Startseite und Tisch lädt keine neue Seite, er ändert nur einen State in `App` (01/5 «SPA Planning Poker App»).

3. Was liefert der Server in Block 1–4 der Vorlesung, was liefert eurer? In den Folien liefert der Server nur den Client-Code aus («Gib mir Datei xyz», 02/6). Unserer liefert zusätzlich eine API mit JSON. Im Produktivbetrieb macht er beides: `express.static` für die Dateien (`server/src/app.ts:24-26`) und die Routen unter `/api`.

4. Was steht in einem HTTP-Request und in einer Response? Request: Methode, Pfad, Header (z. B. `Content-Type`), optional ein Body. Response: Statuscode, Header, optional ein Body (02/10, 02/14). Bei uns ist der Body immer JSON.

5. Welche HTTP-Methoden benutzt ihr, und seit welcher HTTP-Version gibt es sie? GET gibt es schon seit HTTP/0.9, POST kam mit HTTP/1.0, PUT und DELETE mit HTTP/1.1 (02/10-11). Welche wofür: Frage 23.

6. Wo seht ihr die Requests im Browser? DevTools (F12) → Network. Dort sieht man alle 1.5 s ein `GET …?participantId=…` vom Tisch und alle 3 s ein `GET /api/sessions` von der Startseite, mit Statuscode und JSON-Antwort (02/20).

7. Wo seht ihr den gespeicherten Namen? DevTools → Application → Local Storage, Schlüssel `scrum-poker:name` (02/20, `client/src/components/UserProvider.tsx:13`).

8. HTML, CSS, JavaScript: wer macht was? HTML gibt die Struktur (deklarativ), CSS das Aussehen (deklarativ), JavaScript bzw. TypeScript das Verhalten (imperativ, eventbasiert), 02/16. In React schreiben wir HTML als TSX in den Komponenten.

9. Was ist das DOM? Der Baum, den der Browser aus der HTML-Seite baut. JavaScript kann ihn ändern (02/18). React ändert das DOM für uns: Wir ändern nur State, React rechnet aus, was im DOM anders sein muss.

10. Warum sagt ihr «Client» und «Server», wenn beides auf eurem Laptop läuft? Es sind zwei getrennte Programme mit eigener Aufgabe: Der Server hält die Daten und Regeln, der Client zeigt sie an. Dass beide auf `localhost` laufen, ist nur beim Entwickeln so. Im Betrieb läuft der Server auf einem anderen Rechner, die Browser der Teammitglieder sind die Clients.

A.2 Node.js, npm, pnpm (03)

11. Was ist Node.js? Eine JavaScript-Laufzeitumgebung ohne Browser (03/4). Sie führt unseren Server aus und die Werkzeuge des Clients (Vite, TypeScript, oxlint).

12. Was steht in einer package.json? Name, Version, Scripts, Abhängigkeiten (03/10). Bei uns haben `client/` und `server/` je eine eigene, denn «jedes package.json signalisiert ein Projekt» (03/10, `server/package.json:10-27`).

13. Was ist der Unterschied zwischen dependencies und devDependencies? `dependencies` braucht die App beim Ausführen (Server: nur `express`, Client: `react`, `react-dom`). `devDependencies` braucht nur, wer entwickelt: TypeScript, oxlint, Prettier, die Type-Pakete (`server/package.json:18-27`), im Client zusätzlich Vite.

14. Was bedeuten `^5.3.0` und `~6.0.3`? Semantic Versioning (03/10): `^` erlaubt neuere Minor- und Patch-Versionen (5.x.x), `~` nur Patch-Versionen (6.0.x). Die genauen installierten Versionen stehen im Lockfile.

15. Wozu das Lockfile `pnpm-lock.yaml`? Es hält die exakten Versionen aller Pakete fest, auch der indirekten. So installiert jeder Rechner genau dasselbe (wie `package-lock.json`, 03/10). Es wird nie von Hand geändert.

16. Warum pnpm und nicht npm? Die Kursumgebung schreibt pnpm 12 vor (03/17), es ist schnell und spart Platz mit Hard-Links (03/14). Festgelegt mit `"packageManager": "pnpm@12.11.2"` (`server/package.json:6`).

17. Was passiert bei `pnpm dev` im Ordner server/? pnpm startet das Script `dev`, also `node --watch src/index.ts` (`server/package.json:11`). Node führt die TypeScript-Datei direkt aus und startet bei jeder gespeicherten Änderung neu.

18. Wie kann Node eine .ts-Datei ausführen, ohne dass ihr kompiliert? Node 24 kann Type Stripping: Es löscht beim Laden alle Type-Angaben und führt den Rest als JavaScript aus. Das geht, weil wir nur TypeScript benutzen, das man einfach weglassen kann (`erasableSyntaxOnly`, `server/tsconfig.json:20`). Die Typprüfung macht `pnpm build` mit `tsc`.

19. In den Folien wird `"type": "module"` gelöscht. Warum habt ihr es? Das Tutorial benutzt `require('figlet')` (CommonJS, 03/19-20). Wir schreiben überall `import`/`export` wie ab Folie 07/8, dafür braucht Node `"type": "module"` (`server/package.json:5`).

A.3 Server: Express und REST (01/4, 02/4, 02/10-11)

20. Was ist Express, und steht es in den Folien? Eine Library für Web-Server in Node: Sie nimmt HTTP-Requests an und ruft je nach Methode und Pfad unsere Funktion auf. In den Folien nur genannt (01/4 «minimales Backend mit Express.js», 02/4, 09/37 «keine Libraries ausser Express»), erklärt wird es erst im zweiten Kursteil.

21. Was ist Middleware? Nennt eure. Eine Funktion, die Express mit `app.use` in eine Kette hängt. Jede Anfrage läuft der Reihe nach durch diese Kette, bis eine Funktion antwortet; jede kann die Anfrage verändern, beantworten oder weitergeben. Bei uns: `express.json()` (`server/src/app.ts:19`), der Router, `express.static` im Produktivbetrieb, die 404-Middleware und der Error-Handler (`server/src/app.ts:29-43`).

22. Was macht `express.json()`? Sie liest den Body jeder Anfrage mit `Content-Type: application/json`, wandelt den JSON-Text in ein Objekt um und legt es in `req.body` (`server/src/app.ts:18-19`). Ohne diesen Header bleibt `req.body` in Express 5 `undefined`, dann melden unsere Prüfungen z. B. «Name fehlt.». Kaputtes JSON wird zum `SyntaxError`, unser Error-Handler macht daraus 400.

23. Welche Methode für welche Route, und warum? GET liest (Liste, Tisch-Zustand). POST erstellt etwas mit einer ID vom Server (Session, Teilnehmer) und löst Aktionen aus (aufdecken, neue Runde). PUT setzt einen Wert vollständig (Stimme, Deck), idempotent. DELETE entfernt (Session, Teilnehmer). Tabelle in Teil 02, Kapitel 3.

24. Was bedeuten 200, 201, 204, 400, 404, 409, 500? 200 OK mit Daten, 201 Created (Session oder Teilnehmer neu), 204 No Content (gelöscht, kein Body), 400 Bad Request (Body ungültig), 404 Not Found (Session, Teilnehmer oder Route unbekannt), 409 Conflict (abstimmen nach dem Aufdecken), 500 Fehler im Server. Folie 02/10 nennt den Statuscode, die einzelnen Codes stehen nicht in den Folien.

25. Was ist REST? Ein Stil für Web-APIs: Jede Ressource hat eine Adresse (`/api/sessions/abc123/participants`), die HTTP-Methode sagt, was passiert, Daten gehen als JSON hin und her, und der Statuscode sagt, ob es geklappt hat. Nicht in den Folien.

26. Was ist `req.params`, was `req.query`, was `req.body`? `req.params`: Platzhalter im Pfad (`:sessionId`). `req.query`: Werte nach dem `?` (`participantId`). `req.body`: der JSON-Inhalt, nach `express.json()`. Alle drei in der Route zum Abfragen eines Tisches (`server/src/routes/sessions.ts:59-81`) bzw. beim Beitreten.

27. Woran erkennt Express den Error-Handler? An genau vier Parametern `(err, req, res, next)` (`server/src/app.ts:34`). Darum steht `_next` da, obwohl er nicht benutzt wird.

28. Warum steht nach `res.status(404).json(…)` ein `return`? Damit die Route aufhört. Sonst liefe der Code weiter, obwohl die Antwort schon weg ist. Eine zweite Antwort gibt einen Fehler («Cannot set headers after they are sent»), z. B. `server/src/routes/sessions.ts:50-53`. In diesem Beispiel würde schon `session.id` danach abstürzen, weil `session` `undefined` ist; TypeScript meldet das ohne `return` bereits beim Build.

29. Was heisst In-Memory, und was ist der Nachteil? Die Daten liegen nur im Arbeitsspeicher, in einem Array (`server/src/store.ts:33`). Einfach und schnell, aber nach einem Neustart ist alles weg, und es geht nur mit einem einzigen Server-Prozess.

30. Was ist ein Port, und warum 3000 und 5173? Eine Nummer, unter der ein Programm auf dem Rechner erreichbar ist (03/9 zeigt `localhost:3000`). 3000 ist unser Express-Server (`server/src/config.ts:5`), 5173 der Standard-Port von Vite. Ändern lässt sich der Server-Port mit der Umgebungsvariable `PORT`.

A.4 HTML und CSS (04)

31. Welche semantischen Elemente benutzt ihr? `header`, `main` (`client/src/components/App.tsx:24-27`), `section` (z. B. die Karten der Startseite), `ul`/`li` für Listen, echte `button`-Elemente, `label` zum Eingabefeld. Die Folien nennen `main` und `section` als semantische Elemente (04/5) und `ul`/`li` (04/8); `header` und `label` stehen nicht darin.

32. Wie verbindet ihr CSS und Komponenten? Jede Komponente importiert ihre eigene CSS-Datei (`import './HomePage.css'`, 08/19) und setzt Klassen mit `className` (04/9: in React `className` statt `class`).

33. Wie heissen eure CSS-Klassen, und warum so? Mit Präfix der Komponente, BEM-artig: `home-page__card`, `seat__card--covered`. So stören sich Komponenten nicht gegenseitig (04/41, 08/19). Zwei Bindestriche markieren eine Variante.

34. Was macht `box-sizing: border-box`? `width` und `height` schliessen Padding und Rahmen mit ein (Box-Modell, 04/19-21). Steht als Mini-Reset oben in `client/src/index.css:13-17` (05/8).

35. Flexbox oder Grid – wo benutzt ihr was? Grid für die zwei Spalten der Startseite (`client/src/components/HomePage.css:8-11`), Flexbox für alles in einer Richtung: Plätze am Tisch, Karten, Buttons (`client/src/components/PokerTable.css:5-9`). Faustregel: Flexbox eine Richtung, Grid Zeilen und Spalten (04/24-44).

36. Welche Pseudoklassen benutzt ihr? `:hover` für Buttons und `:focus` für den sichtbaren Rahmen bei Tastaturbedienung (`client/src/index.css:55-60`), beide aus 04/14. `:disabled` für gesperrte Buttons steht nicht in den Folien.

37. Was ist `gap` bei Flexbox? Abstand zwischen den Elementen, ohne `margin` an jedem Element. In den Folien gibt es nur `column-gap`/`row-gap` bei Grid; `gap` funktioniert in allen aktuellen Browsern auch bei Flexbox.

38. Wie entsteht der ovale Tisch? Ein Rechteck mit sehr grossem `border-radius: 999px` ab 768 px (`client/src/components/PokerTable.css:60`), dazu ein dicker brauner Rahmen und grüner Hintergrund (04/15, Venn-Übung 04/51).

A.5 Responsive Design (05)

39. Wie macht ihr die App responsive? Mobile First: Die Grundregeln gelten für das Handy, eine Media Query `@media (min-width: 768px)` ergänzt Änderungen für grössere Bildschirme, z. B.

zwei Spalten auf der Startseite (`client/src/components/HomePage.css:49-54`). Dazu Flexbox und Grid (05/3-9).

40. Was ist eine Media Query? Eine CSS-Regel, die nur unter einer Bedingung gilt, z. B. ab einer Bildschirmbreite (05/7). Die Folien zeigen `max-width`, wir schreiben Mobile First mit `min-width`. Beides ist eine Media Query.

41. Wozu `order` im Tisch-CSS? Auf dem Handy kommt der grüne Tisch nach den Plätzen (`order: 1`), ab 768 px steht er wieder in der Mitte (`order: 0`, `client/src/components/PokerTable.css:54`). Folie 05/11.

42. Wie testet ihr die Handy-Ansicht? DevTools → Mobile View (`Ctrl+Shift+M`, 05/5) mit 360 px, 768 px und 1280 px Breite. Es darf nie horizontal scrollen.

43. Was macht `<meta name="viewport">`? Sagt dem Handy-Browser, dass die Seite so breit ist wie der Bildschirm und nicht verkleinert werden soll (`client/index.html:6`, 04/6). Ohne sie tut der Handy-Browser so, als wäre der Bildschirm etwa 980 px breit: Die Media Query ab 768 px greift dann, und man sieht die verkleinerte Desktop-Ansicht.

A.6 TypeScript Basics (06)

44. Was ist TypeScript, und was bringt es euch? JavaScript mit statischen Typen, wird vor dem Ausführen geprüft (06/3). TypeScript meldet z. B., wenn wir vergessen, dass `getSession()` `undefined` zurückgeben kann, oder ein Deck falsch schreiben.

45. `const` oder `let`? `const` als Standard, `let` nur, wenn der Wert sich ändert (06/5-6), z. B. `let port = 3000` in `server/src/config.ts:5`, weil die Umgebungsvariable ihn überschreiben kann. Nie `var`.

46. Was ist ein Union Type, was ein Literal Type? Beispiel? `'fibonacci' | 'tshirt'` ist eine Union aus zwei Literal Types: Nur genau diese zwei Texte sind erlaubt (06/23, `client/src/types/poker.ts:4`). `CardValue | null` ist eine Union mit `null` (06/14).

47. Warum `type` und nicht `interface`? Beides beschreibt die Form eines Objekts. Die Folien empfehlen `type` (06/22), und 09/37 verlangt eine einheitliche Wahl. Darum überall `type`, auch `type Props`.

48. Warum `===` und nicht `==`? `==` wandelt Typen um (`'1' == 1` ist wahr), `===` nicht (06/18). Wir benutzen nur `===` und `!==`.

49. Was gibt `find` zurück, wenn nichts passt? Wo nutzt ihr das? `undefined` (06/16). `getSession` gibt darum `Session | undefined` zurück, und jede Route prüft das (`server/src/store.ts:94-96`).

50. Was ist der Unterschied zwischen `filter`, `find` und `map`? `filter` gibt alle passenden Elemente als neues Array, `find` das erste passende oder `undefined` (06/16), `map` ein neues Array mit umgewandelten Elementen (06/38), z. B. Sessions zu `<SessionListItem>`.

51. Was macht `Number('5')`, und was `Number('?')`? Wandelt Text in eine Zahl um: 5. Bei `'?'` kommt `NaN` (Not a Number, 06/10). So erkennt `toNumber`, welche Karten zum Durchschnitt zählen (`client/src/utils/poker.ts:66-71`).

52. Was ist ein Template-String? Ein Text in Backticks mit eingesetzten Werten: `` `Der Name darf höchstens ${MAX_NAME_LENGTH} Zeichen lang sein.` `` (06/9, `server/src/rules.ts:25`).

A.7 TypeScript Advanced (07)

53. Was ist der Unterschied zwischen Named und Default Export? Named: `export function App()`, Import mit `{ App }` und genau diesem Namen (07/10). Default: `export default`

`…`, beim Import frei benennbar (07/11). In `src/` benutzen wir nur Named Exports. Default Exports kommen von Libraries (`import express from 'express'`, `import react from '@vitejs/plugin-react'`), und `client/vite.config.ts` muss selbst `export default` benutzen, weil Vite die Einstellungen so erwartet.

54. Was macht `import type`, und warum müsst ihr es benutzen? Importiert nur einen Type, der beim Ausführen wegfällt. Mit `verbatimModuleSyntax` aus der Vorlage ist es Pflicht (07/12, `server/tsconfig.json:12`).

55. Wo benutzt ihr Generics? Bei `useState`, wenn der Startwert den Type nicht verrät: `useState<string | null>(null)` in `client/src/components/App.tsx:11` (07/16, 08/32). Ohne Generic wüsste TypeScript nur «immer `null`».

56. Was ist Destructuring? Beispiel? Werte aus Arrays oder Objekten in Variablen auspacken (07/19-21): `const [activeSessionId, setActiveSessionId] = useState(…)` und Props in der Parameterliste `function CardDeck({ deck, selected, disabled, onCardClick }: Props)`.

57. Wo benutzt ihr den Ternary-Operator? Für Klassen und Texte im TSX, z. B. die gewählte Karte: `value === selected ? 'playing-card playing-card--selected' : 'playing-card'` (`client/src/components/CardDeck.tsx:25-29`, 07/15).

58. Was macht der Spread-Operator, und wo würdet ihr ihn brauchen? `[...liste, neu]` bzw. `{ ...objekt, name }` macht eine flache Kopie (07/23-24). Bei uns kommt er nicht vor: Neue Listen entstehen mit `filter` (`client/src/components/HomePage.tsx:100-102`), neue Objekte baut der Server.

A.8 React Basics (08)

59. Was ist eine Komponente? Eine Funktion, die TSX zurückgibt und einen Teil der Oberfläche beschreibt (08/14-15). Sie bekommt Daten über Props und kann eigenen State haben. Keine Klassen-Komponenten (08/14, 09/37).

60. Was ist der Unterschied zwischen Props und State? Props kommen von der Eltern-Komponente und sind nur lesbar (08/16). State gehört der Komponente selbst, wird mit dem Setter geändert und löst ein neues Rendern aus (08/22-24).

61. Was heisst «Lifting State Up»? Beispiel? State liegt in der ersten gemeinsamen Eltern-Komponente aller, die ihn brauchen (08/26). Beispiel: Der Tisch-Zustand liegt in `PokerRoom`, weil `PokerTable`, `CardDeck`, `DeckSwitcher` und `RoundResult` ihn brauchen. Der State steht in `client/src/components/PokerRoom.tsx:36`.

62. Was ist ein Callback via Props? Die Eltern-Komponente gibt eine Funktion als Prop nach unten, das Kind ruft sie auf, um etwas nach oben zu melden (08/25-27): `onCardClick`, `onJoin`, `onLeave`, `onNameChange`.

63. Wozu die `key`-Prop? Damit React beim Ändern einer Liste weiss, welches Element welches ist (08/31). Wir nehmen stabile IDs, z. B. `key={session.id}` (`client/src/components/HomePage.tsx:158`), nicht den Index: Der Index verschiebt sich, wenn ein Eintrag wegfällt.

64. Was ist Conditional Rendering, was Conditional Styling? Rendering: je nach Bedingung andere Komponente oder nichts, mit `&&`, Ternary oder `if` mit `return` (08/7, 08/37), z. B. HomePage oder PokerTablePage in `App`. Styling: je nach Bedingung eine andere Klasse (08/36), z. B. `seat--own` für den eigenen Platz.

65. Was ist `children`, und wo nutzt ihr es? Alles zwischen öffnendem und schliessendem Tag kommt als Prop `children` an (08/17). `PokerTable` zeigt so die Tischmitte, die `PokerRoom`

hineinlegt; der `UserProvider` umschliesst die App.

66. Warum darf man State nicht direkt ändern? React merkt eine Änderung nur über den Setter mit einem **neuen** Wert (08/28, 08/33). Würde man das alte Array verändern und zurückgeben, rendert React nicht neu. Darum `filter` beim Löschen eines Listeneintrags.

67. Was ist ein kontrolliertes Eingabefeld? Der Wert kommt aus dem State (`value={name}`), jede Eingabe geht über `onChange` zurück (`client/src/components/NameForm.tsx:34-35`). React ist die einzige Quelle für den Inhalt.

68. Wie lest ihr den Wert aus einem `<select>`? Im Handler mit `ChangeEvent<HTMLSelectElement>` aus `event.target.value` (08/42, `client/src/components/DeckSwitcher.tsx:16-24`). Weil das nur ein `string` ist, suchen wir den passenden `DeckType` in `DECK_TYPES`.

69. Was macht StrictMode? Er startet in der Entwicklung jeden Effekt zweimal (Effekt, Cleanup, Effekt) und ruft jede Komponente doppelt auf, damit fehlende Cleanups sofort auffallen (08/24, 09/3, `client/src/main.tsx:16-22`). Im Produktiv-Build läuft jeder Effekt nur einmal.

A.9 React Hooks (09)

70. Was macht `useEffect`, und wann läuft er? Er führt Code nach dem Rendern aus, für alles ausserhalb von React (Server, Timer). Er läuft beim Mount und wenn sich ein Wert im Dependency Array ändert, `[]` heisst nur beim Mount (09/11-13).

71. Warum braucht es ein Cleanup? Wo habt ihr welche? Was ein Effekt startet, muss beim Unmount aufhören (09/15), sonst gibt es ein Memory Leak (09/30). Drei Stellen: Polling der Liste (`client/src/components/HomePage.tsx:70-73`), Polling am Tisch (`client/src/components/PokerRoom.tsx:76-79`) und das Verlassen der Session im Cleanup des Beitritts (`client/src/components/PokerTablePage.tsx:63-70`).

72. Warum ist der Effekt-Callback nicht direkt `async`? Ein Effekt darf nur eine Cleanup-Funktion zurückgeben, eine `async`-Funktion gibt aber immer ein Promise zurück (09/23). Darum eine innere `async function` und sie sofort aufrufen (09/24).

73. Was macht das `ignore`-Flag? Jeder Effekt-Durchlauf hat ein eigenes `ignore`, das Cleanup setzt es auf `true`. Eine Antwort, die danach noch ankommt, wird nicht mehr in den State geschrieben (`client/src/components/PokerTablePage.tsx:34`). Das Muster stammt aus der React-Dokumentation.

74. Was ist ein Custom Hook? Habt ihr einen? Eine Funktion mit Namen `use…`, die selbst Hooks benutzt (09/26). Ja: `useLocalStorage` (`client/src/hooks/useLocalStorage.ts:12-29`), wie in der Hooks-Übung 09/32.

75. Was heisst `useState(() => …)`? Lazy Initial Value: Die Funktion läuft nur beim ersten Render (09/8). So wird `localStorage` nur einmal gelesen und nicht bei jedem Tastendruck (`client/src/hooks/useLocalStorage.ts:14-20`).

76. Was heisst `setSessions((currentSessions) => …)`? Der Setter bekommt eine Funktion und ruft sie mit dem neusten State auf (09/7). Sicher, auch wenn inzwischen ein Poll eine neue Liste gesetzt hat (`client/src/components/HomePage.tsx:95`).

77. Wie funktioniert euer Polling? Im `useEffect` startet `window.setInterval` die Ladefunktion alle 1.5 s (Tisch) bzw. 3 s (Liste), das Cleanup stoppt sie mit `window.clearInterval` (09/29-30). Genau das Muster `useAutoCounter` aus 09/30 («Endpoint regelmässig abfragen»).

78. Was ist `fetch`, und wann wirft es einen Fehler? Die Browser-Funktion für HTTP-Requests (09/22). Sie wirft nur, wenn gar keine Antwort kommt (Server aus, offline). Eine 404 ist für `fetch`

ein Erfolg, darum prüfen wir `response.ok` und `response.status` selbst (`client/src/api/sessions.ts:108-113`).

79. Was macht `async`/`await` mit `try`/`catch`? `await` wartet auf ein Promise, ohne den Browser zu blockieren. Fehler fängt `catch` (09/17). Jeder Server-Aufruf in den Komponenten steht in `try`/`catch` und zeigt eine Meldung. Ausnahme: Das Verlassen hängt nur `.catch(() => {})` an und zeigt nichts, weil der Heartbeat-Timeout die Person ohnehin entfernt (`client/src/components/PokerTablePage.tsx:68`).

80. Was ist Context, und steht er in den Folien? Ein Weg, einen Wert an alle Komponenten darunter zu geben, ohne ihn durch jede Ebene als Prop weiterzureichen. Nicht in den Folien 1–9; die Aufgabenstellung verlangt ihn für den Namen. `createContext` in `client/src/context/UserContext.ts:12-15`, gelesen mit `useContext(UserContext)`.

Teil B – Fragen zu unserem Projekt

B.1 «Zeig mir, wo …»

81. … der Wechsel zwischen Startseite und Pokertisch passiert. `client/src/components/App.tsx:11` (State `activeSessionId`) und `client/src/components/App.tsx:17-20` (bedingtes Rendering). Gesetzt wird der State über den Callback `onJoin`, zurückgesetzt in `leaveSession` (`client/src/components/App.tsx:13-15`). Pflicht seit 07/6 und 09/36.

82. … der Name gespeichert wird. `client/src/hooks/useLocalStorage.ts:23-26`: `setValue` setzt den State und schreibt in den `localStorage`. Benutzt vom `UserProvider` (`client/src/components/UserProvider.tsx:13`).

83. … die Stimmen vor dem Aufdecken geheim bleiben. Auf dem Server, in `toSessionState` (`server/src/store.ts:39-48`). Nur wenn aufgedeckt ist oder es die eigene Stimme ist, wird der Wert mitgeschickt. Der Client bekommt fremde Stimmen also nie zu früh, auch nicht im Network-Tab.

84. … ein Request an den Server geschickt wird. Nur in `client/src/api/sessions.ts`, in der Funktion `send` (`client/src/api/sessions.ts:21-38`). Komponenten rufen `fetch` nie selbst auf, sondern Funktionen wie `castVote`.

85. … der Server die Eingaben prüft. `server/src/validation.ts`, z. B. `getJoinError` (`server/src/validation.ts:10-19`), aufgerufen in der Route zum Beitreten (`server/src/routes/sessions.ts:90-94`).

86. … Karteileichen entfernt werden. Timer in `server/src/index.ts:9`, Aufräumen in `server/src/store.ts:142-152`, Lebenszeichen in der GET-Route (`server/src/routes/sessions.ts:74-79`).

87. … der Durchschnitt berechnet wird. Im Client, `computeResult` in `client/src/utils/poker.ts:89-125`, aufgerufen von `RoundResult` (`client/src/components/RoundResult.tsx:25`). Der Server schickt nur die Stimmen.

88. … automatisch aufgedeckt wird. `revealIfEveryoneVoted` in `server/src/store.ts:165-175`, aufgerufen nach jeder Stimme und wenn jemand geht.

89. … die Liste der Sessions alle 3 Sekunden geladen wird. `client/src/components/HomePage.tsx:44-74`, das Intervall steht in `SESSION_LIST_POLL_INTERVAL_MS` (`client/src/components/HomePage.tsx:20`).

90. … das Deck gewechselt wird, vom Klick bis zum Server. `<select>` mit `updateDeck` (`client/src/components/DeckSwitcher.tsx:16-24`) → `handleDeckChange` in `PokerRoom` → `changeDeck` in `client/src/api/sessions.ts:174-189` → Route `server/src/routes/sessions.ts:148-163` → `changeDeck` im Store (`server/src/store.ts:188-195`).

B.2 «Was passiert, wenn …»

91. … der Server während einer Runde abstürzt? Beim Entwickeln antwortet der Vite-Proxy mit 502, weil er Express nicht erreicht (ohne Proxy bekäme `fetch` gar keine Antwort und würde werfen). `api/sessions.ts` macht in beiden Fällen daraus «Server nicht erreichbar.» (`client/src/api/sessions.ts:43-49`). `PokerRoom` zeigt die Meldung über dem letzten Stand und pollt weiter. Startet der Server neu, sind die Daten weg: Die nächste Antwort ist 404, und der Client zeigt «Diese Session existiert nicht (mehr)».

92. … jemand den Tab schliesst, ohne «Verlassen» zu klicken? Kein DELETE kommt an. Die Polls fehlen, `lastSeen` wird alt, und der Server entfernt die Person nach höchstens 20 Sekunden (15 s Timeout + bis 5 s bis zum nächsten Aufräumen).

93. … man am Tisch die Seite neu lädt (F5)? Man landet auf der Startseite, nur der Name bleibt (07/6). Die Session-ID war normaler State in `App`. Der alte Platz verschwindet nach höchstens 20 s, wer weitermachen will, tritt neu bei.

94. … zwei Personen gleichzeitig das Deck wechseln? Node bearbeitet die Requests nacheinander. Der zweite gewinnt, beide sehen beim nächsten Poll denselben Zustand.

95. … jemand per curl eine Karte schickt, die es nicht gibt? `getVoteError` findet sie nicht in `DECK_CARDS` → 400 «Diese Karte gibt es im aktuellen Deck nicht.» (`server/src/validation.ts:30-37`).

96. … jemand nach dem Aufdecken noch abstimmt? 409 Conflict (`server/src/routes/sessions.ts:129-133`). Im Client sind die Karten nach dem Aufdecken ohnehin gesperrt.

97. … man die gewählte Karte nochmals anklickt? `handleCardClick` schickt `null`, die Stimme wird zurückgezogen (`client/src/components/PokerRoom.tsx:145-152`).

98. … jemand die Session löscht, während andere am Tisch sitzen? Ihr nächster Poll bekommt 404, `getSession` gibt `null` zurück, `PokerRoom` stoppt das Intervall und zeigt «Diese Session existiert nicht (mehr)» (`client/src/components/PokerRoom.tsx:53-58`).

99. … man StrictMode entfernt? Effekte laufen auch in der Entwicklung nur einmal. Die App funktioniert gleich, aber ein fehlendes Cleanup würde man nicht mehr sofort bemerken. Wegen StrictMode gibt es den Code, der den überzähligen Teilnehmer gleich wieder abmeldet (`client/src/components/PokerTablePage.tsx:40-47`).

100. … der Tab länger im Hintergrund liegt? Chrome bremst Timer in Hintergrund-Tabs nach etwa fünf Minuten auf einen Aufruf pro Minute. Dann fehlen die Polls, und der Server entfernt die Person nach 15–20 s. Kommt man zurück, steht «Du bist nicht mehr am Tisch» (`client/src/components/PokerRoom.tsx:111-125`). Bekannte Grenze, in der Dokumentation Kapitel 8 beschrieben.

B.3 «Warum habt ihr … statt …?»

101. … den Ansichtswechsel mit State statt mit React Router gelöst? Weil der geänderte Leistungsnachweis genau das verlangt: Pflicht §3.1 «Wechsel zwischen Home und Pokertisch über State», Router ist nur noch optional (07/6, 09/36). State ist einfacher und kommt ohne zusätzliche

Library aus. Folge: kein teilbarer Link, nach einem Reload bleibt nur der Name, auch das sagt 07/6.

102. … keine Error Boundary? Eine Error Boundary gibt es in React nur als Klasse, und 09/37 sagt «keine Class-Components». Fehler von Server-Aufrufen fangen wir überall mit `try`/`catch` ab. Ein Fehler beim Rendern wäre ein Programmierfehler; dagegen helfen strenges TypeScript, `pnpm lint` und Ausprobieren.

103. … keine automatischen Tests im Projekt? Tests kommen in den Folien nicht vor, und Test-Libraries wie Vitest wären zusätzliche Libraries (09/37). Wir prüfen mit `pnpm lint`, `pnpm format:check`, `pnpm build` (Typprüfung) in beiden Ordnern und klicken alle Abläufe in zwei Browserfenstern durch. Ehrlich dazu: Die KI hat ausserhalb der Abgabe einen automatischen Browser-Test benutzt, um ihre Änderungen zu prüfen (Dokumentation Kapitel 12).

104. … Prettier und oxlint? 09/37: Prettier mit «Format on Save» und `pnpm lint` ausführen. oxlint ist der Linter der Projektvorlage, der Server benutzt denselben (`server/package.json:14-16`).

105. … Node 24 und pnpm 12? Die Folien schreiben genau diese Umgebung vor (03/17, 07/4). Node 24 kann ausserdem `.ts`-Dateien direkt ausführen, darum braucht der Server kein Zusatzwerkzeug.

106. … zwei Projekte und die Types doppelt statt eines gemeinsamen Ordners? Der Client ist die Projektvorlage (07/4), ein reines React-Projekt. Der Server ist ein eigenes Node-Projekt («Jedes package.json signalisiert ein Projekt», 03/10). Ein gemeinsamer Ordner bräuchte einen pnpm-Workspace, der nicht in den Folien vorkommt. Nachteil: Types, Kartenlisten und Namensregel stehen zweimal (`server/src/rules.ts:1-3` verweist auf die Kopie im Client).

107. … Polling statt WebSockets? Polling steht in den Folien (09/30), passt zu REST und braucht keine Library. Nachteil: bis 1.5 s Verzögerung und viele kleine Requests. Für ein Team von 5–10 Personen kein Problem, und das Polling dient gleich als Heartbeat.

108. … Context für den Namen statt Props? Die Aufgabenstellung verlangt einen Context für den Namen. Bei zwei Ebenen ginge es auch mit State in `App` und Props. Context ist nicht in den Folien, wir haben ihn aus der React-Dokumentation.

109. … `unknown` in validation.ts statt `any`? Mit `any` würde TypeScript gar nichts prüfen. Mit `unknown` müssen wir den Body zuerst mit `typeof` und `in` prüfen, bevor wir ihn benutzen. Daten von aussen sind nie vertrauenswürdig.

110. … die Prüfung im Server, obwohl der Client schon prüft? Prüfung im Client ist Komfort (sofortige Meldung), Prüfung im Server ist Sicherheit: Jeder kann mit `curl` an der Oberfläche vorbei Requests schicken.

B.4 «Was würdet ihr ändern?»

111. Was würdet ihr mit mehr Zeit verbessern? Eine Datenbank statt des Arrays, damit ein Neustart nichts löscht. WebSockets statt Polling für sofortige Updates. Eine Rolle «Moderator», damit nicht jede Person aufdecken und löschen kann.

112. Was gefällt euch selbst am Code nicht? Die doppelten Types und Regeln in Client und Server (Frage 106). Und der Error-Handler macht aus seltenen Fehlern wie einem zu grossen Body eine 500 statt des passenden Codes (Teil 02, Kapitel 7).

113. Wie würdet ihr Tests einführen, wenn sie erlaubt wären? Für Store und `computeResult` einfache Unit-Tests (reine Funktionen ohne React und Express), für die Routen Tests mit echten HTTP-Requests, und ein Browser-Test für den Ablauf mit zwei Personen.

114. Wie würdet ihr die Hintergrund-Tab-Grenze lösen? Das Timeout im Server deutlich erhöhen oder beim Zurückkehren in den Tab automatisch neu beitreten. Beides ändert die Strategie gegen Karteileichen und wäre gut zu überlegen.

115. Was würdet ihr an der Struktur ändern, wenn die App wächst? React Router für teilbare Links (dann optionale Erweiterung §3.4), ein Ordner pro Seite, und im Server die Teilnehmer-Routen in einen eigenen Router, damit `routes/sessions.ts` nicht zu lang wird.

B.5 KI-Nutzung

Antwortet hier **ehrlich und ruhig**. Die Nutzung ist in der Dokumentation (Kapitel 15) offengelegt. Erfindet nichts, was ihr nicht gemacht habt.

116. Habt ihr KI benutzt? Ja, und zwar für den grössten Teil des Codes und der Dokumentation (Claude Code von Anthropic). Wir haben die Aufgaben formuliert, die Anforderungen aus Aufgabenstellung und Folien vorgegeben, die Ergebnisse geprüft und Änderungen verlangt. Das steht offen in der Dokumentation, Kapitel 15.

117. Wie stellt ihr sicher, dass ihr den Code versteht? Wir haben verlangt, dass der Code vereinfacht und an die Folien angeglichen wird, damit wir jede Zeile erklären können. Dann haben wir den Code mit einer Zeile-für-Zeile-Erklärung durchgearbeitet, uns gegenseitig abgefragt und Übungen gemacht: Code ändern, beobachten, zurücksetzen. *(Nur sagen, was ihr wirklich gemacht habt.)*

118. Welche Teile habt ihr selbst geschrieben? Ehrlich antworten, was stimmt. Wenn nichts von Hand geschrieben wurde: «Geschrieben hat die KI. Unsere Arbeit war, vorzugeben, was gebaut wird, zu prüfen, Fehler zu melden und zu entscheiden, z. B. den Code zu vereinfachen und auf das Niveau der Folien zu bringen.» Die Gruppe trägt das in Kapitel 15 der Dokumentation ein.

119. Warum kommen Dinge vor, die nicht in den Folien stehen (Express-Details, Context, `unknown`)? Express und Context verlangt die Aufgabenstellung, `unknown` braucht es für sichere Daten von aussen. Wir haben die KI angewiesen, sonst nur Kursstoff zu benutzen, und jede Ausnahme in Dokumentation und Leitfaden markieren und begründen lassen. Erklären können wir sie trotzdem.

120. Würdet ihr wieder so arbeiten? Eine ehrliche, eigene Antwort. Zum Beispiel: «KI spart Tipparbeit, aber man muss jede Zeile verstehen, sonst kann man nichts ändern und nichts erklären. Darum haben wir den Code vereinfachen lassen und ihn Zeile für Zeile gelernt.»